Forecasting Cockroach Search Interest in New Orleans

A time series analysis of Google Trends data, 2016 to 2026

Nuzhat Faizah

2026-10-22

Table of contents

1 Introduction 1
2 Data 2
3 Data preparation 3
4 Exploration 5
5 Weather 9
6 Model comparison 11
7 Forecast to August 2029 15
8 Limitations 22
9 Conclusion 22
References 23
Software 23

1 Introduction

This project began the way many New Orleans stories do: with a cockroach in the kitchen. After a move to the city in August 2026, the first month in a new apartment brought nightly sightings, a few sandal battles, and a search history full of pest control advice. Somewhere between losing those battles and hiding the food in airtight containers, a different question appeared. Does New Orleans have a predictable roach season, and can data forecast it? As it turns out, the city may have four seasons: winter, spring, roach, and fall.

Cockroaches, besides being my least favorite unsolicited guests, are a public health concern. Cockroach allergens are an established asthma trigger, and children in urban housing with both cockroach allergy and high exposure have more asthma related hospital visits (Rosenstreich et al., 1997). Cockroaches can also carry bacteria such as Salmonella and E. coli from drains and garbage onto food and kitchen surfaces (Donkor, 2020). Beyond physical health, infestations can cause stress and lost sleep, heavy use of indoor sprays adds pesticide exposure, and persistent infestations often signal leaks, cracks, and poor maintenance in housing. Exposure is also common in New Orleans. In the 2015 American Housing Survey, 42.1 percent of households in the New

Orleans metropolitan area reported seeing cockroaches in the past year, the highest share among 25 large metropolitan areas (U.S. Census Bureau, 2015). Cockroach exposure in New Orleans is therefore both widespread and relevant to health.

No public source tracks cockroach infestations month by month, so the project uses a proxy. Google Trends reports how often people in a region search for a term, on a relative scale from 0 to 100. Monthly search interest for "cockroach" in the New Orleans metropolitan area serves as a measure of public attention to cockroaches. The measure reflects people noticing cockroaches and looking for help, not the number of infestations. All results in the report should be read with the distinction in mind.

The analysis addresses three questions. The first question asks whether search interest follows a stable seasonal pattern and how that pattern has changed since 2016. The second question asks whether warmer or wetter months bring more searches than usual for their time of year. The third question asks what search interest is likely to look like through August 2029, and how uncertain that forecast is. Together, the three questions move from describing the past to explaining it to projecting the future.

The main findings are previewed here and developed in later sections. Search interest peaks every summer, usually in July, and reaches its low point every December. A change in how Google collected data in January 2022 coincides with a clear increase in the level of the series. Forecasting models reduced typical errors by about 24 percent compared with a simple benchmark, while temperature added little predictive value and rainfall showed no association. The forecast projects July peaks between 76 and 84 through 2029, with wide uncertainty. The report closes with the limitations of search data and the choices behind each result. The cockroaches, unfortunately, were not available for comment.

2 Data

The analysis uses two data sources, both saved to files before the analysis began. Google Trends search interest (Google, 2026) was retrieved on October 7, 2026, for the term "cockroach" in the New Orleans metropolitan area (Google market code US-LA-622), covering January 2016 through September 2026. Daily mean temperature and daily rainfall for central New Orleans came from the Open Meteo historical weather archive for the same period (Open-Meteo, 2026). The report loads both saved files rather than requesting new data, so every rendering of the report uses identical inputs. All code and data are available at https://github.com/nf9925/new_orleans_roach_forecast.

```
# Packages. The report runs in a fresh R session, so it loads its own.
library(tidyverse)   # data handling and plots
library(tsibble)     # time series data frames and yearmonth()
library(feasts)      # diagnostics such as STL and ljung_box
library(fable)       # forecasting models
library(fabletools)  # model(), forecast(), accuracy(), scenarios()
library(ggtime)      # time series plots
```

```
# Every date decision lives here, in one place.
start_month  <- yearmonth("2016 Jan")  # first month after Google's 2016 change
end_month    <- yearmonth("2026 Sep")  # last complete month
test_n       <- 24                     # months held out for testing
step_month   <- yearmonth("2022 Jan")  # Google's data collection change
scenario_end <- yearmonth("2029 Aug")  # last forecast month
train_end    <- end_month - test_n     # last training month

# Saved data files. No calls to Google or the weather service.
roach_df <- read_csv("data/roach_raw_pulled_2026_10_07.csv",
                     show_col_types = FALSE)
wx_daily <- read_csv("data/weather_daily_pulled_2026_10_07.csv",
                     show_col_types = FALSE)
```

Google Trends values need careful interpretation. Each value shows search volume relative to the busiest month in the requested window, which Google sets to 100. Google also builds each retrieval from a sample of searches and reports 0 when volume falls below a privacy threshold. The window was therefore limited to 2016 onward, so a noisy month from the sparse early years could not define the scale. As a result, a value of 50 means half the search interest of the peak month, not 50 searches.

The weather data come from a reanalysis product rather than a single weather station. Reanalysis combines a weather model with observations on a regular grid, which gives complete daily records without gaps. Monthly averages from reanalysis closely follow station records, though individual days can differ. For a monthly analysis, the reanalysis data provide a consistent and complete record.

3 Data preparation

The first preparation step converts the search data into a monthly time series and checks it for problems. A time series object records the time order of the data and confirms that no month is missing. The checks below confirm the length of the series, the absence of gaps, and the number of months reported as zero.

```
roach_ts <- roach_df %>%
  mutate(month = yearmonth(date)) %>%
  filter(month >= start_month, month <= end_month) %>%
  select(month, hits) %>%
  as_tsibble(index = month)

tibble(
  check  = c("Months in the series", "Missing months",
             "Months reported as 0"),
  result = c(nrow(roach_ts), sum(has_gaps(roach_ts)$.gaps),
```

```
            sum(roach_ts$hits == 0))
) %>%
  knitr::kable()
```

check	result
Months in the series	129
Missing months	0
Months reported as 0	1

The series starts in January 2016 for two reasons. Before 2011, Google assigned searches to locations less accurately, and the New Orleans series shows almost only zeros in those years. Google also changed its data collection system on January 1, 2016, and again on January 1, 2022, as noted on the Google Trends website. Starting in 2016 places every month on the same side of the 2016 change and leaves one known change, in 2022, inside the series. The 2022 change is handled directly in the models.

```
# Months reported as 0, recorded for the report.
imputed_months <- roach_ts %>% filter(hits == 0) %>% pull(month)

# Treat zeros as missing on the log scale, fit a seasonal ARIMA,
# and let the model estimate the missing month from its
# neighbors and the usual seasonal pattern.
roach_log <- roach_ts %>%
  mutate(log_hits = if_else(hits == 0, NA_real_, log(hits))) %>%
  select(month, log_hits)

roach_ts <- roach_log %>%
  model(ARIMA(log_hits)) %>%
  interpolate(roach_log) %>%
  mutate(hits = exp(log_hits)) %>%
  select(month, hits)

imputed_value <- roach_ts %>%
  filter(month %in% imputed_months) %>%
  pull(hits) %>%
  round(1)

# Step variable: 0 before January 2022, 1 from January 2022 onward.
roach_ts2 <- roach_ts %>%
  mutate(step_2022 = if_else(month >= step_month, 1, 0))
```

One month required imputation. The retrieval reported December 2024 as 0, while every other December in the series fell roughly between 17 and 41. A sudden zero surrounded by normal months is consistent with Google's privacy threshold or sampling noise, not an absence of searches. The zero also prevents the log transformation used later, since the log of 0 is undefined. The value was treated as missing and estimated as 30.9 with a seasonal ARIMA model on the log scale.

4 Exploration

The series shows a strong and regular seasonal pattern. Search interest rises each spring, peaks in summer, and falls to its lowest level in winter. The level of the series also rises after 2022, and summer peaks after 2022 are higher than before. The highest value in the series, 100, occurs in August 2023.

```
roach_ts %>%
  autoplot(hits) +
  # One label per year, so the axis stays readable on the PDF page.
  scale_x_yearmonth(date_breaks = "1 year", date_labels = "%Y") +
  labs(title = "Cockroach search interest, New Orleans",
       x = "Month", y = "Search interest (0 to 100)") +
  theme_minimal()
```

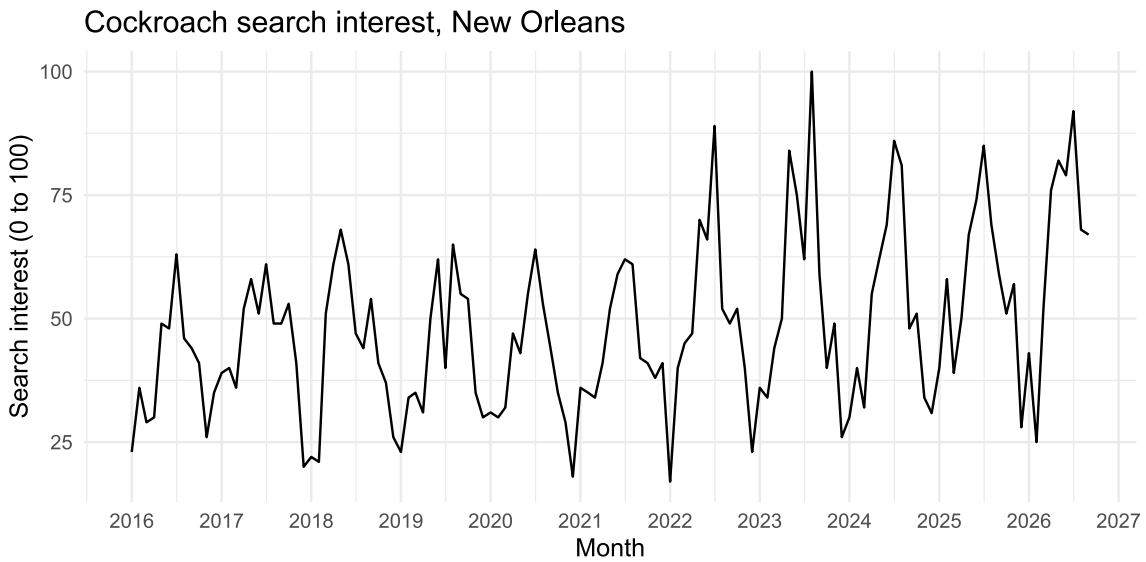


Figure 1: Monthly search interest for cockroach, New Orleans, 2016 to 2026.

The seasonal plot places each year on the same January to December axis. Every year follows the same shape, with low values from December to March, a rise in April, a peak from June to August, and a decline through the fall. Years after 2022 sit above earlier years, especially in summer. The timing of the season is stable, while its intensity has grown.

```r
# The same plot as in the analysis script, with each year's label
# placed at the end of its line. The figure is drawn at its original
# size, then scaled down as a whole picture to fit the page, so the
# labels keep the same spacing as the original.
roach_ts %>%
  gg_season(hits, labels = "right") +
  labs(title = "Seasonal plot: each line is one year",
       y = "Search interest", x = "Month")
```

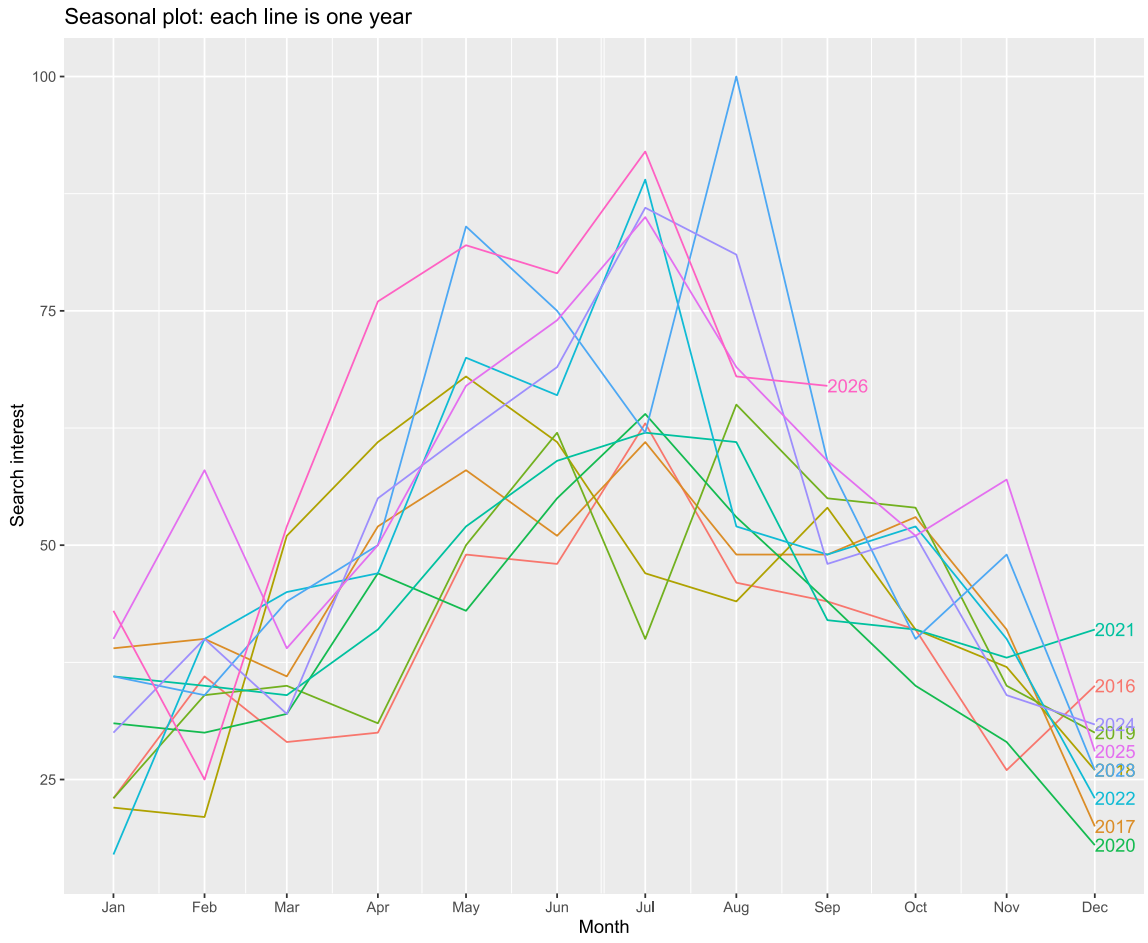


Figure 2: Seasonal plot. Each line shows one year.

The subseries plot groups each calendar month across all years. Summer months show a clear upward movement over time, while December stays roughly flat. Summer values have therefore grown faster than winter values. A pattern where seasonal swings grow with the level of the series is called multiplicative seasonality.

```
# The same plot as in the analysis script. The figure is drawn at
# its original size (9.6 x 7.9 inches), then scaled down as a whole
# picture to fit the page, so the layout stays identical and the
# labels keep the same spacing as the original.
roach_ts %>%
  gg_subseries(hits) +
  labs(title = "Subseries plot by calendar month",
       y = "Search interest")
```

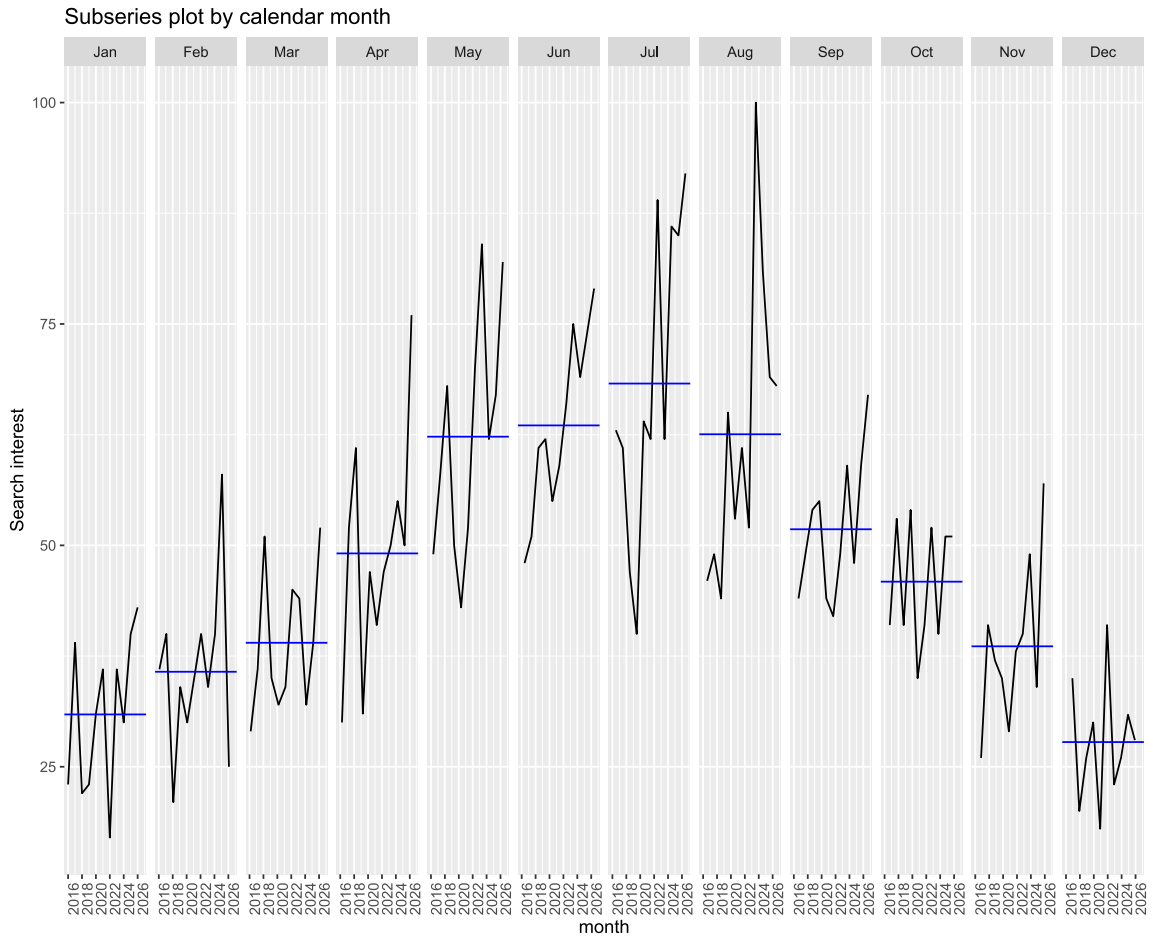


Figure 3: Subseries plot. Blue lines show the average for each calendar month.

A decomposition separates the series into trend, seasonal, and remainder components. The trend stays near 40 to 45 through 2021, rises after 2022, and continues to climb through 2026. The seasonal component swings about 40 points between the winter low and the summer high. The remainder is mostly small noise, with its largest value in August 2023.

```
roach_ts %>%
  model(STL(hits ~ season(window = "periodic"))) %>%
  components() %>%
  autoplot() +
  labs(title = "STL decomposition of cockroach search interest") +
  theme_minimal()
```

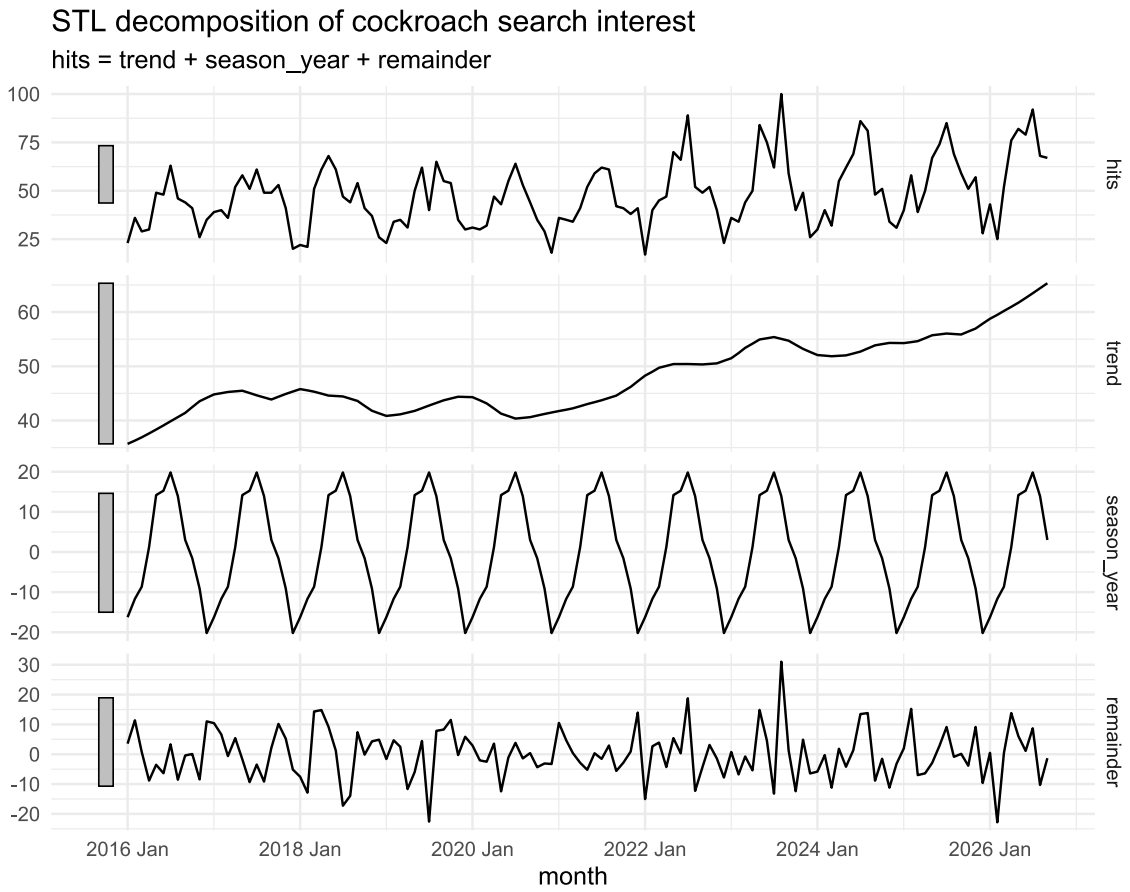


Figure 4: STL decomposition of search interest with a fixed seasonal pattern.

The log transformation makes the seasonal swings more even. On the log scale, a fixed distance represents a fixed percentage, so growing peaks become similar in size. The logged series shows summers consistently far above winters across all years, with a typical July about 2.3 times a typical December. All models in the report therefore work on the log of search interest.

```
roach_ts %>%
  mutate(log_hits = log(hits)) %>%
  autoplot(log_hits) +
  scale_x_yearmonth(date_breaks = "1 year", date_labels = "%Y") +
```

```
  labs(title = "Logged cockroach search interest",
       x = "Month", y = "log(search interest)") +
  theme_minimal()
```

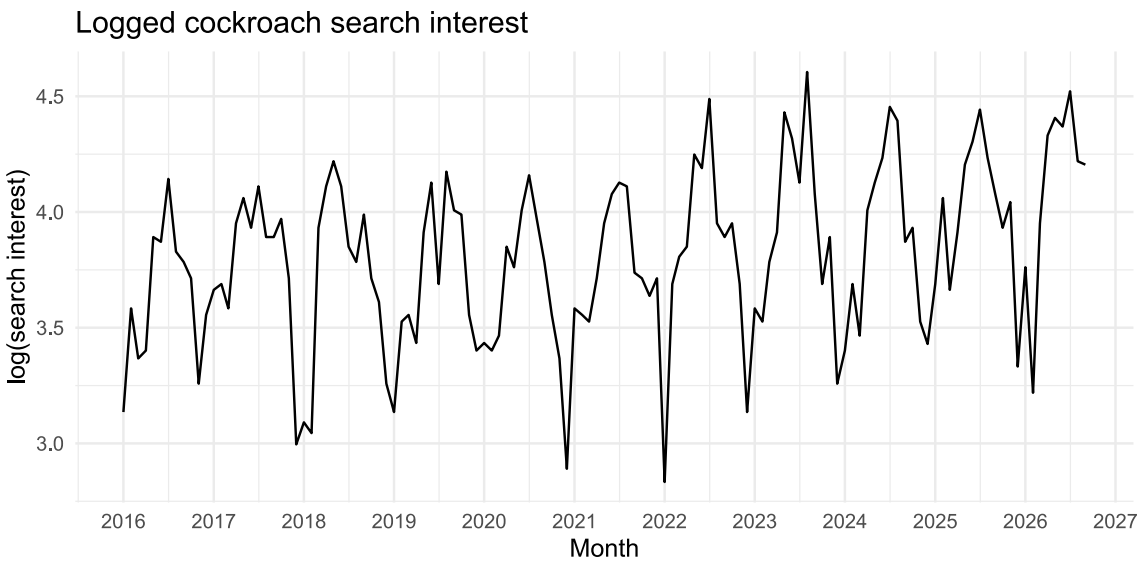


Figure 5: Logged search interest.

5 Weather

Weather was tested as a possible driver of unusual search activity. Temperature and season move together, since summer is both hot and the peak of cockroach season. A model with both raw temperature and seasonality cannot separate their effects. The analysis therefore uses temperature anomalies, defined as the difference between a month's mean temperature and the average for that calendar month.

```
# Monthly weather from daily records.
wx_monthly <- wx_daily %>%
  mutate(month = yearmonth(date)) %>%
  group_by(month) %>%
  summarise(temp_mean    = mean(temp, na.rm = TRUE),
            precip_total = sum(precip, na.rm = TRUE),
            heavy_rain_days = sum(precip >= 1, na.rm = TRUE),
            .groups = "drop")

# Normal temperature for each calendar month, from training
# months only, so no information from the test period leaks in.
wx_normals <- wx_monthly %>%
  filter(month <= train_end) %>%
```

```
  mutate(cal_month = lubridate::month(as.Date(month))) %>%
  group_by(cal_month) %>%
  summarise(temp_norm = mean(temp_mean), .groups = "drop")

wx_anom <- wx_monthly %>%
  mutate(cal_month = lubridate::month(as.Date(month))) %>%
  left_join(wx_normals, by = "cal_month") %>%
  mutate(temp_anom = temp_mean - temp_norm) %>%
  select(month, temp_anom)

roach_ts3 <- roach_ts2 %>%
  left_join(wx_anom, by = "month")
```

Rainfall was examined before any modeling. Heavy rain could push cockroaches out of flooded drains, while dry spells could push them indoors in search of water. The test compared the number of days with at least one inch of rain against the deseasonalized remainder of search interest. Monthly rainfall totals were not used, since one downpour and steady drizzle can produce the same total.

```
roach_remainder <- roach_ts %>%
  model(STL(log(hits) ~ season(window = "periodic"))) %>%
  components() %>%
  as_tibble() %>%
  select(month, remainder)

rain_check <- roach_remainder %>%
  left_join(wx_monthly, by = "month")

rain_cor <- round(cor(rain_check$heavy_rain_days,
                      rain_check$remainder), 2)
rain_cutoff <- round(2 / sqrt(nrow(rain_check)), 2)
```

```
ggplot(rain_check, aes(x = heavy_rain_days, y = remainder)) +
  geom_jitter(width = 0.15, alpha = 0.6) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  labs(title = "Heavy rain days vs roach search surprises",
       x = "Days with 1 inch of rain or more",
       y = "Remainder (log scale)") +
  theme_minimal()
```

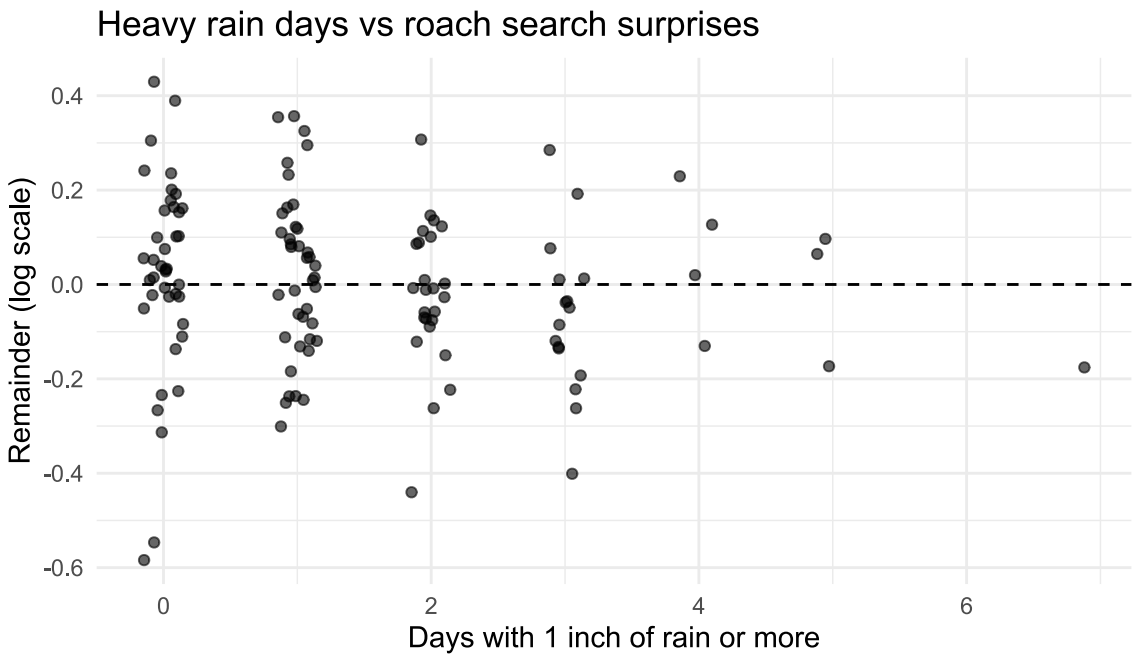


Figure 6: Heavy rain days compared with deseasonalized search interest. Each point is one month.

Rainfall showed no meaningful association with search interest. The correlation between heavy rain days and the remainder was −0.13, below the rough threshold of 0.18 needed to stand out from random noise with 129 months. Months with no heavy rain showed both the largest increases and the largest decreases in search interest. Rainfall was therefore not included in the forecasting models.

6 Model comparison

The models were compared on data they had not seen, following standard practice in forecasting (Hyndman and Athanasopoulos, 2021). The last 24 months, from October 2024 to September 2026, were held out as a test period, and each model was trained only on earlier months. Four models were compared. Seasonal naive repeats the value from the same month of the previous year and serves as a benchmark. ETS (exponential smoothing) weights recent observations more heavily, and two regression models with ARIMA errors include the 2022 step, one with temperature anomalies and one without.

```
roach_train <- roach_ts3 %>% filter(month <= train_end)
roach_test  <- roach_ts3 %>% filter(month >  train_end)

# Both ARIMA models share one error structure, so temperature
# is the only difference between them.
roach_fit <- roach_train %>%
  model(
    snaive     = SNAIVE(log(hits)),
```

```
    ets        = ETS(log(hits)),
    arima_step = ARIMA(log(hits) ~ step_2022 +
                         pdq(0, 0, 0) + PDQ(2, 1, 0)),
    arima_temp = ARIMA(log(hits) ~ step_2022 + temp_anom +
                         pdq(0, 0, 0) + PDQ(2, 1, 0))
  )

# Forecast the test period with the observed weather (best case).
roach_fc <- roach_fit %>% forecast(new_data = roach_test)

# December 2024 is imputed, so it is left out of the scoring.
test_acc <- roach_fc %>%
  filter(month != yearmonth("2024 Dec")) %>%
  accuracy(roach_ts3) %>%
  select(.model, MAE, RMSE, MAPE) %>%
  arrange(MAE)

knitr::kable(test_acc, digits = 2,
           caption = "Accuracy on the test period, excluding the imputed month.")
```

.model	MAE	RMSE	MAPE
ets	8.46	10.88	15.50
arima_temp	9.19	11.13	17.48
arima_step	9.32	11.42	17.36
snaive	9.79	11.56	19.40

```
ets_test_mae    <- round(test_acc$MAE[test_acc$.model == "ets"], 2)
snaive_test_mae <- round(test_acc$MAE[test_acc$.model == "snaive"], 2)
```

ETS was the most accurate model on the test period. Its mean absolute error (MAE) was 8.46 points, compared with 9.79 for the seasonal naive benchmark. MAE measures the average size of a miss, RMSE gives extra weight to large misses, and MAPE expresses the average miss as a percentage. The imputed December 2024 value falls inside the test period and was excluded from scoring; including it did not change the ranking.

```
# gg_tsresiduals() combines three panels into one figure, and a
# title added with labs() may land on a single panel or fail, so
# this figure relies on its caption instead.
roach_fit %>%
  select(ets) %>%
```

```
  gg_tsresiduals()

lb <- augment(roach_fit) %>%
  filter(.model == "ets") %>%
  features(.innov, ljung_box, lag = 24)

lb_stat <- round(lb$lb_stat, 1)
lb_p    <- round(lb$lb_pvalue, 2)
```

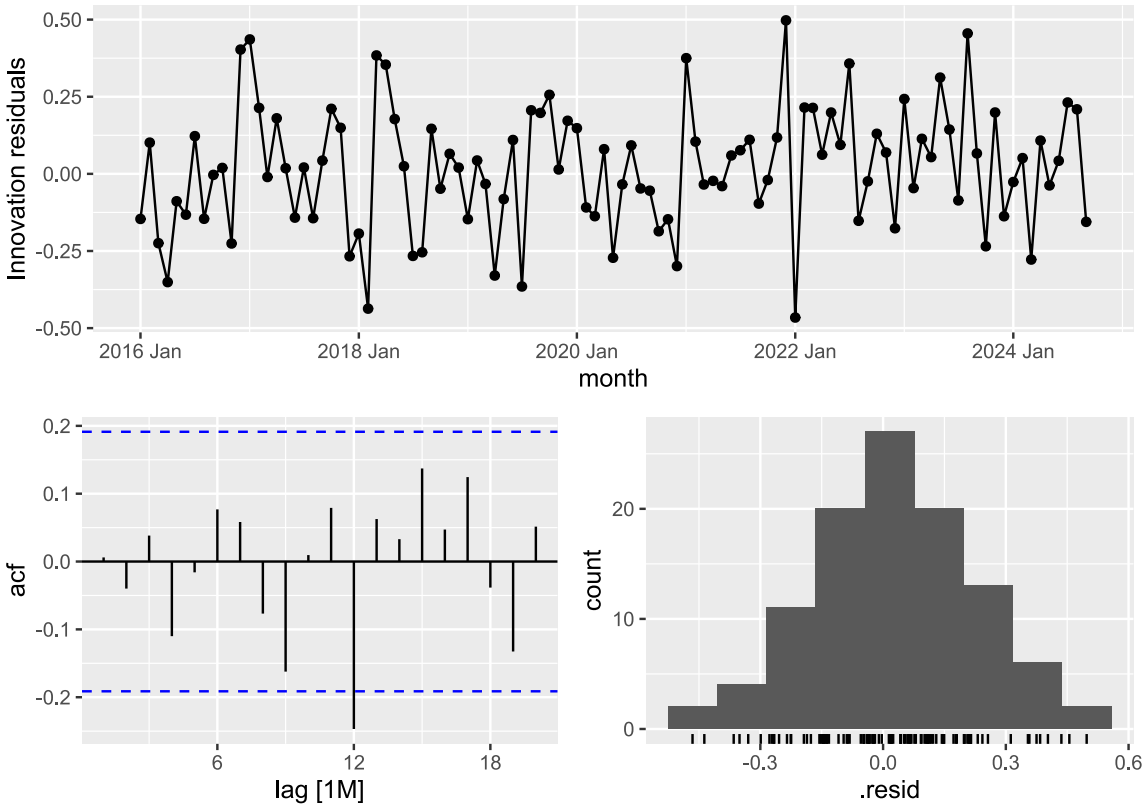


Figure 7: Residual diagnostics for the ETS model.

The ETS residuals were checked for patterns the model failed to capture. A Ljung Box test across 24 lags showed no significant overall autocorrelation (Q = 25.4, p = 0.38). The autocorrelation at lag 12, however, fell outside the significance bound. A negative value at lag 12 indicates that a surprise in one month tends to reverse in the same month of the following year, a small seasonal pattern the fixed seasonal component of ETS cannot capture.

A single test period can favor one model by chance, so the comparison was repeated with time series cross validation. The first training window covered January 2016 to December 2022, so the step variable had a full year of post 2022 data. Each later window added three months, and each model forecast the following 12 months from every window. Averaging across all windows gives a more reliable comparison than a single test.

```
roach_cv <- roach_ts3 %>%
  stretch_tsibble(.init = 84, .step = 3)

cv_fit <- roach_cv %>%
  model(
    snaive     = SNAIVE(log(hits)),
    ets        = ETS(log(hits)),
    arima_step = ARIMA(log(hits) ~ step_2022),
    arima_temp = ARIMA(log(hits) ~ step_2022 + temp_anom)
  )

cv_future <- new_data(roach_cv, 12) %>%
  left_join(roach_ts3 %>%
              as_tibble() %>%
              select(month, step_2022, temp_anom),
            by = "month") %>%
  filter(!is.na(step_2022))

cv_fc <- cv_fit %>% forecast(new_data = cv_future)

cv_acc <- cv_fc %>%
  filter(month != yearmonth("2024 Dec")) %>%
  accuracy(roach_ts3) %>%
  select(.model, MAE, RMSE, MAPE) %>%
  arrange(MAE)

n_windows <- n_distinct(cv_future$.id)
```

```
knitr::kable(cv_acc, digits = 2,
             caption = "Cross validation accuracy, excluding the imputed month.")
```

.model	MAE	RMSE	MAPE
arima_step	8.67	11.45	15.46
ets	8.68	11.78	15.69
arima_temp	8.78	11.19	15.85
snaive	11.38	14.97	22.32

```
cv_three  <- cv_acc %>% filter(.model != "snaive")
cv_low    <- round(min(cv_three$MAE), 1)
cv_high   <- round(max(cv_three$MAE), 1)
```

```
cv_snaive <- round(cv_acc$MAE[cv_acc$.model == "snaive"], 1)
cv_gain   <- round(100 * (1 - mean(cv_three$MAE) / cv_snaive))
```

Cross validation showed no clear winner among the three main models. Across 15 forecast windows, ETS and both regression models had MAE values between 8.7 and 8.8, and each accuracy measure favored a different model by a small margin. All three outperformed the seasonal naive benchmark (MAE 11.4), reducing typical errors by about 24 percent. Adding temperature did not consistently improve accuracy, even though the temperature model received the observed weather. The advantage of ETS on the single test period was therefore partly a matter of chance.

7 Forecast to August 2029

The main forecast uses the regression model with ARIMA errors and the 2022 step, without temperature. The model tied for best accuracy in cross validation, and it needs no future weather, since the step stays at 1 after 2022. The model was refit on all 129 months before forecasting. Its ARIMA orders were chosen automatically, as in cross validation.

```
roach_main_fit <- roach_ts3 %>%
  model(arima_step = ARIMA(log(hits) ~ step_2022))

main_coefs <- tidy(roach_main_fit)
knitr::kable(main_coefs %>% select(term, estimate, std.error),
             digits = 3,
             caption = "Coefficients of the main forecasting model.")
```

term	estimate	std.error
sar1	−0.301	0.165
sar2	−0.157	0.148
sma1	−0.817	0.212
step_2022	0.122	0.062
intercept	0.017	0.010

```
step_pct  <- round(100 * (exp(main_coefs$estimate[main_coefs$term == "step_2022"]) - 1))
drift_pct <- round(100 * (exp(main_coefs$estimate[main_coefs$term == "intercept"]) - 1), 1)

h <- as.integer(scenario_end - end_month)

future_main <- new_data(roach_ts3, h) %>%
  mutate(step_2022 = 1)
```

```
roach_main_fc <- roach_main_fit %>%
  forecast(new_data = future_main)

main_df <- roach_main_fc %>%
  mutate(median_fc = median(hits)) %>%
  hilo(level = 80) %>%
  unpack_hilo(`80%`) %>%
  as_tibble() %>%
  select(month, mean_fc = .mean, median_fc,
         lower = `80%_lower`, upper = `80%_upper`)

july_fc <- main_df %>%
  filter(lubridate::month(as.Date(month)) == 7)

jul_first <- round(july_fc$median_fc[1])
jul_last  <- round(july_fc$median_fc[nrow(july_fc)])
n_above   <- sum(main_df$upper > 100)
max_upper <- round(max(main_df$upper))
```

The model estimates a step of about 13 percent in January 2022 and an upward drift of about 1.7 percent per year. The drift term is weakly estimated, so the gradual rise in forecast peaks should be read as an assumption carried forward rather than a confident finding. The step reflects Google's data collection change and possibly real changes in behavior, which the data cannot separate.

```r
# The same plot as in the analysis script. It sits on its own
# landscape page, so the 80 monthly labels get the full page width
# and the text stays readable.

key_points <- main_df %>%
  filter(lubridate::month(as.Date(month)) %in% c(7, 12)) %>%
  mutate(
    label     = paste0(format(as.Date(month), "%b %Y"), ": ",
                       round(median_fc)),
    vjust_val = if_else(lubridate::month(as.Date(month)) == 7, -1.2, 2)
  )

hist_df <- roach_ts3 %>%
  as_tibble() %>%
  filter(month >= yearmonth("2023 Jan")) %>%
  select(month, hits)

forecast_start <- as.Date(end_month + 1)

ggplot() +
  geom_ribbon(data = main_df,
              aes(x = as.Date(month), ymin = lower, ymax = upper),
              fill = "grey80") +
  geom_line(data = hist_df,
            aes(x = as.Date(month), y = hits),
            color = "black") +
  geom_line(data = main_df,
            aes(x = as.Date(month), y = median_fc),
            color = "steelblue") +
  geom_point(data = key_points,
```

```
            aes(x = as.Date(month), y = median_fc),
            color = "steelblue", size = 2) +
  geom_text(data = key_points,
            aes(x = as.Date(month), y = median_fc,
                label = label, vjust = vjust_val),
            size = 3) +
  geom_vline(xintercept = forecast_start, linetype = "dotted") +
  annotate("text", x = forecast_start, y = 15,
           label = paste("Forecast starts",
                         format(forecast_start, "%b %Y")),
           hjust = -0.05, size = 3) +
  geom_hline(yintercept = 100, linetype = "dashed") +
  scale_x_date(breaks = seq(as.Date("2023-01-01"),
                            as.Date(scenario_end),
                            by = "1 month"),
               date_labels = "%b %y",
               # Small padding on the left, more on the right, so the
               # last label (Jul 2029) is not cut off at the edge.
               expand = expansion(mult = c(0.01, 0.04))) +
  scale_y_continuous(breaks = seq(0, 110, by = 10)) +
  labs(title = "Cockroach search interest forecast to August 2029",
       subtitle = "Black: actual data. Blue: median forecast. Grey band: 80% prediction interval. Dashed line:
index maximum (100)",
       x = "Month", y = "Search interest (0 to 100)") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1,
                                   size = 7),
        panel.grid.minor.x = element_blank())
```

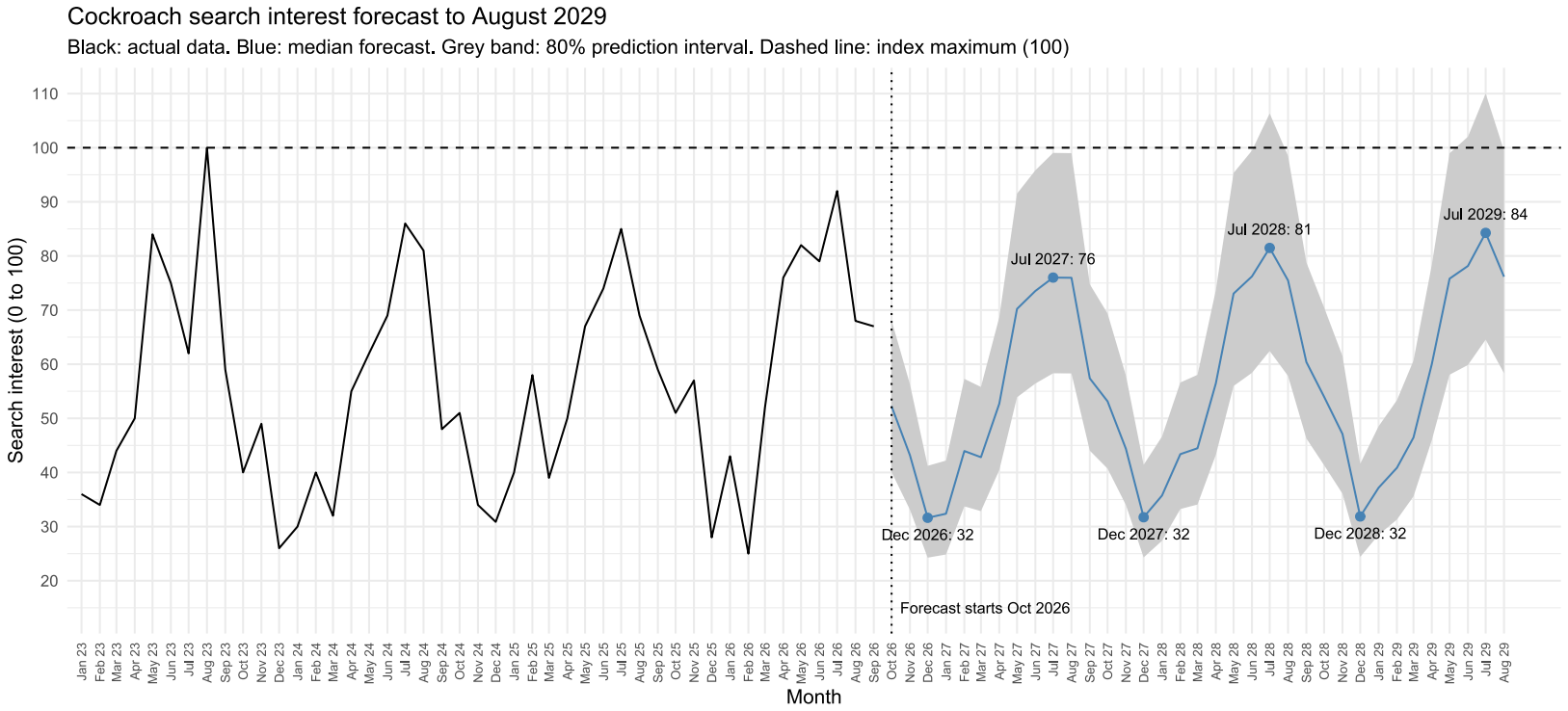


Figure 8: Forecast of cockroach search interest to August 2029.

```
july_fc %>%
  mutate(month = format(month)) %>%
  select(month, median_fc, lower, upper) %>%
  knitr::kable(digits = 1,
               col.names = c("Month", "Median forecast",
                             "80% lower", "80% upper"),
               caption = "Forecast for July of each year.")
```

Month	Median forecast	80% lower	80% upper
2027 Jul	76.0	58.3	99.0
2028 Jul	81.5	62.4	106.3
2029 Jul	84.3	64.5	110.0

The forecast continues the seasonal pattern through August 2029. Forecasts are reported as medians, since the log scale makes each forecast distribution skewed and pushes the mean slightly above the typical value. Median July forecasts rise from 76 in 2027 to 84 in 2029, and every December falls to about 32. The 80 percent prediction interval widens with each year, and its upper edge exceeds the index maximum of 100 in 3 months, reaching 110. Values above 100 are impossible for the Google Trends index, so the upper edge of the interval should be read as “very high” rather than literally.

July gets the spotlight because it is the peak month of roach season, so its forecast shows how intense each future season is expected to be.

A sensitivity check asked how much temperature would change the forecast if its estimated effect were real. A regression model with ARIMA errors, the 2022 step, and temperature anomalies was refit on all data. Forecasts were then compared under two scenarios: normal temperatures, and June to September months 3°F warmer than normal.

```
roach_temp_fit <- roach_ts3 %>%
  model(arima_temp = ARIMA(log(hits) ~ step_2022 + temp_anom))

temp_eff <- tidy(roach_temp_fit) %>%
  filter(term == "temp_anom") %>%
  transmute(
    pct  = round(100 * (exp(3 * estimate) - 1), 1),
    low  = round(100 * (exp(3 * (estimate - 1.96 * std.error)) - 1), 1),
    high = round(100 * (exp(3 * (estimate + 1.96 * std.error)) - 1), 1)
  )

future_temp <- new_data(roach_ts3, h) %>%
  mutate(step_2022 = 1,
```

```
         cal_month = lubridate::month(as.Date(month)))

roach_scen <- scenarios(
  normal      = future_temp %>% mutate(temp_anom = 0),
  hot_summers = future_temp %>%
    mutate(temp_anom = if_else(cal_month %in% 6:9, 3, 0)),
  names_to = "scenario"
)

scen_july <- roach_temp_fit %>%
  forecast(new_data = roach_scen) %>%
  mutate(median_fc = median(hits)) %>%
  as_tibble() %>%
  filter(lubridate::month(as.Date(month)) == 7) %>%
  select(scenario, month, median_fc) %>%
  pivot_wider(names_from = scenario, values_from = median_fc) %>%
  left_join(main_df %>% select(month, lower, upper), by = "month") %>%
  mutate(gap = hot_summers - normal,
         interval_width = upper - lower)

scen_july %>%
  mutate(month = format(month)) %>%
  select(month, normal, hot_summers, gap, interval_width) %>%
  knitr::kable(digits = 1,
               col.names = c("Month", "Normal", "Hot summers",
                             "Gap", "Width of 80% interval"),
               caption = "July forecasts under two temperature scenarios.")
```

Month	Normal	Hot summers	Gap	Width of 80% interval
2027 Jul	85.6	88.3	2.7	40.7
2028 Jul	86.2	88.9	2.7	43.9
2029 Jul	87.5	90.2	2.7	45.5

```
mean_gap   <- round(mean(scen_july$gap), 1)
min_width  <- round(min(scen_july$interval_width))
max_width  <- round(max(scen_july$interval_width))
temp_jul27 <- round(scen_july$normal[1], 1)
```

Temperature would change the forecast very little. A summer month 3°F warmer than normal was associated with an estimated 3.1 percent increase in search interest, with a 95 percent interval from −1.5 to 8 percent. The interval includes zero, so the data cannot rule out no effect. The gap

between the two scenarios averaged 2.7 points in July, while the 80 percent interval of the main forecast spanned 41 to 45 points. Weather is therefore a minor factor compared with ordinary forecast uncertainty.

The choice of model mattered more than the weather. Under normal temperatures, the temperature model forecast 85.6 for July 2027, while the main model forecast 76. The difference between the two models is several times larger than the effect of a hot summer. Prediction intervals assume the chosen model is correct, so they do not include uncertainty about the model itself.

8 Limitations

Google searches are a stand in for cockroaches, not a cockroach census. Searches capture people noticing cockroaches and asking for help, which is not the same as counting infestations. Longtime residents who have made peace with their uninvited roommates may search less, while newcomers like me may search a lot. Households with the most cockroaches may also have the least internet access. In short, the data measure human panic, and panic has its own patterns.

Google Trends brings its own quirks. Each retrieval draws a sample, so pulling the same data twice can give slightly different numbers, and one month in this retrieval came back as a suspicious 0 and had to be imputed. Google also changed how it collects data in January 2022, right when the series jumps, and the step variable cannot tell whether the jump came from Google's new ruler or from real changes in behavior. Every result here rests on a single retrieval from October 7, 2026, so a future pull may tell a slightly different story.

The models have limits too. The log transformation lets prediction intervals climb above 100, a value the index can never reach, so the top of the grey band is generous in the least welcome way. The rising forecast peaks rest on a weakly estimated drift, and the prediction intervals assume the chosen model is the right one. The cross validation windows overlap, so the fifteen tests are less independent than they sound, and the temperature model got to peek at the real weather during testing. None of the limits breaks the findings, but each one deserves a raised eyebrow.

Finally, the scope is small. The weather comes from a reanalysis grid rather than a backyard thermometer, and the two can differ on individual days. The study covers one city and one search term, so a Houstonian googling "palmetto bug" falls outside its reach. Generalizing beyond New Orleans would need more cities, more search terms, and probably more courage.

9 Conclusion

New Orleans really does seem to have four seasons, and roach season is the most punctual of them. Search interest peaks every summer, usually in July, and bottoms out every December, with a typical July bringing more than twice the searches of a typical December. The level rose after Google's 2022 data collection change, and summer peaks have grown since then. If the cockroaches kept a calendar, it would look exactly like the seasonal plot.

The weather turned out to be a supporting actor, not the star. Hotter than normal months showed a small bump in searches that the data could not separate from zero, and heavy rain showed nothing at all. The forecasting models beat the "same as last year" benchmark by about a quarter, but

giving them the temperature did not make them any smarter. The season itself, not the weather report, runs the show.

July is the busiest month for roaches, so its forecast shows how intense each future roach season will be. For my household, the future holds unwanted screams and sandals slapping the floor. Speaking of the future, the forecast expects the cockroaches to keep their summer appointments through August 2029. Median July forecasts range from 76 to 84, and even a summer 3°F hotter than normal would nudge those peaks only slightly. The timing is highly predictable, even if the exact level is not, which gives health and housing programs a head start: pest prevention and asthma outreach can arrive before the cockroaches do.

On a personal note, the project started with a sandal and ended with a forecast. I still prefer my cockroaches on charts rather than in my kitchen, but at least now I know when to expect them. See you next July.

References

Donkor, E. S. (2020). Cockroaches and food borne pathogens. *Environmental Health Insights, 14*, 1178630220913365. https://doi.org/10.1177/1178630220913365

Google. (2026). *Google Trends.* Retrieved October 7, 2026, from https://trends.google.com

Hyndman, R. J., and Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts. https://otexts.com/fpp3

Open-Meteo. (2026). *Historical Weather API.* Retrieved October 7, 2026, from https://open-meteo.com. Data licensed under CC BY 4.0.

R Core Team. (2026). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. https://www.R-project.org

Rosenstreich, D. L., Eggleston, P., Kattan, M., Baker, D., Slavin, R. G., Gergen, P., Mitchell, H., McNiff Mortimer, K., Lynn, H., Ownby, D., and Malveaux, F. (1997). The role of cockroach allergy and exposure to cockroach allergen in causing morbidity among inner city children with asthma. *New England Journal of Medicine, 336*(19), 1356 to 1363.

U.S. Census Bureau. (2015). *American Housing Survey for the New Orleans Metropolitan Area: 2015.* Washington, DC: U.S. Census Bureau.

Software

The analysis used R version 4.6.0 (R Core Team, 2026) with the packages tidyverse, tsibble, feasts, fable, fabletools, and ggtime. Search data came from Google Trends through the gtrendsR package, and weather data came from the Open Meteo historical weather archive. All code and data are available at https://github.com/nf9925/new_orleans_roach_forecast.